

Syntactic and Semantics errors

L06 - Programming Errors

Department of Computer Science
University of Pretoria

04 March 2024

Overview

- 1 Syntax error
- 2 Semantic error
- 3 Difference between Syntax and Semantics error
 - Common Syntax Errors
- 4 Programming Errors
 - Example 1
 - Example 2
 - Example 3
 - Example 4

Syntax error

Definition: Errors in the structure of your code that violate the rules of the C++ language.

The compiler catches syntax errors and prevents your code from running.

Examples:

Missing semicolons,

Misspelled keywords,

Unbalanced parentheses or braces

Syntax error cnt...

- Syntax doesn't have any relationship with the meaning of the statement;
- Syntax is only associated with the grammar and structure of the programming language.
- Syntax errors are encountered after the program has been executed.
- Some examples of syntax errors include missing semicolons in C++, undeclared variables in C++, although such errors are easy to catch.

Semantic error

Definition: Errors in the logic of your code that cause it to produce incorrect or unexpected results. The compiler cannot detect semantic errors – your code will compile successfully.

Semantic errors are trickier because your code might look correct but still isn't doing what you intend. This is where debugging skills come into play. You need to step through your code's logic carefully to identify where the problem lies.

If there is any semantic error and even when the statement has correct syntax, it wouldn't perform the function that was intended for it to do. Thus, such errors are difficult to catch.

Difference between Syntax and Semantics error

Syntax error: Syntax is one that defines the rules and regulations that helps to write any statement in a programming language.

Semantics error: Semantics is one that refers to the meaning of the associated line of code in a programming language.

Syntax error: Syntax does not have any relationship with the meaning of the statement.

Semantics error: Semantics tells about the meaning.

Difference between Syntax and Semantics error

Syntax error: Syntax errors are encountered after the program has been executed.

Semantics error: They are encountered at runtime.

Syntax error: Syntax errors are easy to catch.

Semantics error: Are difficult to catch.

Common Syntax errors

Missing Semicolon:

Example: `cout << "Hello , world"`

Misspelled Keywords:

Example: `int main() { if (x > 5) cout << "x is greater"; return 0; }`

Unbalanced Parentheses/Braces:

Example: `if (x == 10 { cout << "x is ten" }`

Common Semantic errors

Incorrect Calculations:

Example: `int area = width / height;` (should be multiplication)

Type Mismatches:

Using incompatible data types in operations or assignments.

Example: `int age = "25";` (trying to assign a string to an integer variable)

Uninitialized Variables:

Using a variable before it is assigned a value.

Example: `int x; cout << x;` (x hasn't been assigned a value yet)

Division by Zero:

Attempting to divide by zero, which can lead to program crashes or unexpected behavior.

Example: `int result = 10 / (x - x);` (if x equals x, the denominator becomes zero)

```
// C++ program to demonstrate semantic error
```

```
#include <iostream>
using namespace std;
```

```
// Driver Code
```

```
int main()
```

```
{
```

```
    // Return statement before cout
    return 0;
```

```
    // Print the value
    cout << "GFG!";
```

```
}
```

```
}
```

Output

The output will be blank because the above program is **semantically incorrect syntax**.

This program has **no syntax error as it is following every programming rule** but still, it will not print anything on the screen because the **return statement is written before the cout statement** which causes the program to terminate before printing anything on the screen.

```
// C++ program to demonstrate basic operation
// without any syntax and semantic error

#include <iostream>
using namespace std;

// Driver Code
int main()
{
    // To print gfg
    cout << "GFG!";

    return 0;
}
```

Output

GFG!

This is the correct code i.e, without any syntax and semantic errors.

Important to note on Syntax errors

The first type of error you will encounter when coding are syntax errors.

These occur when the C++ compiler is unable to parse the code because the code violates the expected code structure.

For example, incorrect spelling of a keyword and missing a semi-colon will result in a syntax error. The compiler will try and resolve the errors to the best of its ability. The line where the error occurred may therefore not be at the location as indicated by the compiler error messages.

Identify all the syntax errors in the following C++ program.

```
#inc lud <ios t ream>

using namespace s td

int main ( ) f
cout >> "COS132 2024 " ;

return 0 ;
```

Identify all the errors in the following C++ program.

```
#include <iostream>
using namespace std;

int main() {
    std::cout << " Hello , World!"
    return 0;
}
```


Output

In this example, the semicolon at the end of the cout statement is missing, resulting in a syntax error.

Identify all the errors in the following C++ program.

```
#include <iostream>
using namespace std;

int main() {
    int x = 5;
    int y = 0;
    int z = x / y;
    cout << "Result: " << z;
    return 0;
}
```

Output

In this example, division by zero leads to a semantic error. The code compiles without error, but the program crashes at runtime.